

Differential gene expression

Material

<https://youtu.be/6Y5uCQWRRbg?si=mcV9qJUap1qvFjy3>

- More information on [pseudobulk analysis](#)
- [Muscat](#) for pseudobulk DGE.
- [Paper](#) on the robustness of different differential expression analysis methods

Exercises

Find all markers for each cluster

Load the seu dataset you have created yesterday:

And load the following packages (install them if they are missing):

Code starts here:

```
library(Seurat)
library(edgeR) # BiocManager::install("edgeR")
library(limma)
library(dplyr)
library(scuttle)
```

The function `FindAllMarkers` performs a Wilcoxon plot to determine the genes differentially expressed between each cluster and the rest of the cells. Other types of tests than the Wilcoxon test are available. Check it out by running `?Seurat::FindAllMarkers`.

Now run analysis:

Code starts here:

```
de_genes <- Seurat::FindAllMarkers(seu, min.pct = 0.25,
                                   only.pos = TRUE)
```

Subset the table to only keep the significant genes, and you can save it as a csv file if you wish to explore it further. Then extract the top 3 markers per cluster:

Code starts here:

```
de_genes <- subset(de_genes, de_genes$p_val_adj < 0.05)
write.csv(de_genes,
          "day3/de_genes_FindAllMarkers.csv",
          row.names = F, quote = F)

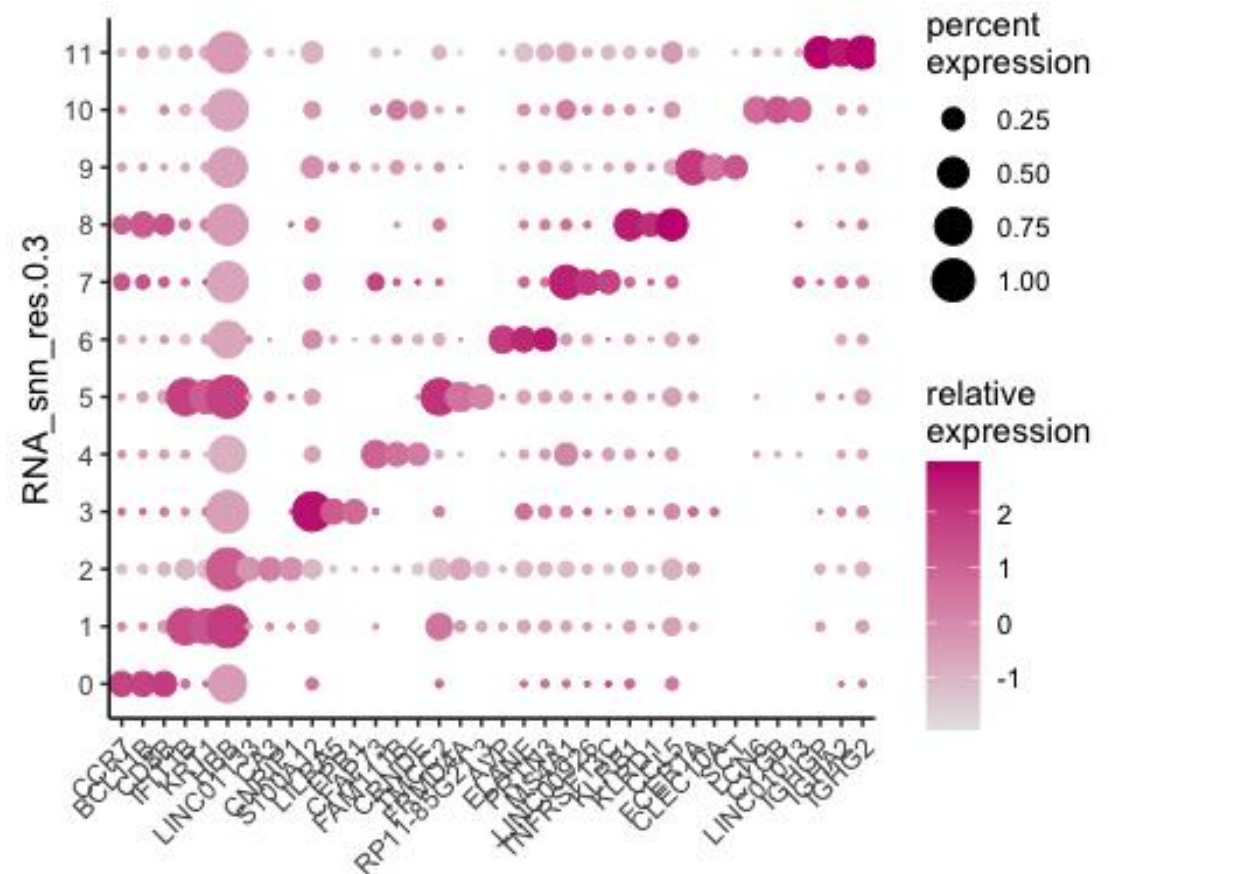
top_specific_markers <- de_genes %>%
```

```
group_by(cluster) %>%  
top_n(3, avg_log2FC)
```

And generate e.g. a dotplot:

Code starts here:

```
dittoSeq::dittoDotPlot(seu,
  vars = unique(top_specific_markers$gene),
  group.by = "RNA_snn_res.0.3")
```



! Exercise

What are significant marker genes in cluster 0 and 8? Are the T cell genes in there?

You can re-load the vector with immune genes with:

Code starts here:

```
tcell_genes <- c("IL7R", "LTB", "TRAC", "CD3D")
```

 Answer

Code starts here:

```
de_genes[de_genes$gene %in% tcell_genes,] |> knitr::kable()
```

	p_val	avg_log2FC	pct.1	pct.2	p_val_adj	cluster	gene
CD3D	0	2.9119600	0.772	0.226	0	0	CD3D
TRAC	0	2.5821466	0.628	0.202	0	0	TRAC
LTB	0	1.8792775	0.761	0.394	0	0	LTB
IL7R	0	2.8925743	0.437	0.114	0	0	IL7R
LTB1	0	1.3839002	0.676	0.465	0	7	LTB
TRAC1	0	2.3960975	0.749	0.274	0	8	TRAC
CD3D1	0	2.1124053	0.795	0.326	0	8	CD3D
LTB2	0	1.8691213	0.757	0.462	0	8	LTB
IL7R1	0	1.9115516	0.468	0.172	0	8	IL7R
LTB3	0	0.9057526	0.743	0.466	0	10	LTB

So, yes, the T-cell genes are highly significant markers for cluster 0 and 8.

Differential expression between groups of cells

The FindMarkers function allows to test for differential gene expression analysis specifically between 2 groups of cells, i.e. perform pairwise comparisons, eg between cells of cluster 0 vs cluster 2, or between cells annotated as T-cells and B-cells.

First we can set the default cell identity to the cell types defined by SingleR:

Code starts here:

```
seu <- Seurat::SetIdent(seu, value = "SingleR_annot")
```

Run the differential gene expression analysis and subset the table to keep the significant genes:

Code starts here:

```
deg_cd8_cd4 <- Seurat::FindMarkers(seu,  
                                   ident.1 = "CD8+ T cells",  
                                   ident.2 = "CD4+ T cells",  
                                   group.by = seu$SingleR_annot,  
                                   test.use = "wilcox")  
deg_cd8_cd4 <- subset(deg_cd8_cd4, deg_cd8_cd4$p_val_adj<0.05)
```

! Exercise

Are CD8A, CD8B and CD4 in there? What does the sign (i.e. positive or negative) mean in the log fold change values? Are they according to the CD8+ and CD4+ annotations? Check your answer by generating a violin plot of a top differentially expressed gene.



Answer

You can check out the results with:

Code starts here:

[View\(deg_cd8_cd4\)](#)

	p_val	avg_log2FC	pct.1	pct.2	p_val_adj
CD8A	0.0e+00	5.8036617	0.336	0.008	0.0000000
CTSW	0.0e+00	3.4118773	0.276	0.030	0.0000000
CCL5	0.0e+00	4.2922622	0.285	0.062	0.0000000
CD8B	0.0e+00	1.3123538	0.470	0.178	0.0000000
NKG7	0.0e+00	4.5376500	0.225	0.037	0.0000000
CST7	0.0e+00	4.4123574	0.143	0.012	0.0000000
GZMA	0.0e+00	3.8501654	0.169	0.025	0.0000000
TRGC2	0.0e+00	3.4931638	0.144	0.018	0.0000000
RPS27	0.0e+00	-0.1820605	1.000	1.000	0.0000000
KLRD1	0.0e+00	4.3589947	0.108	0.008	0.0000000
ID2	0.0e+00	0.8632969	0.565	0.353	0.0000000
GZMK	0.0e+00	3.0113231	0.130	0.022	0.0000000
HCST	0.0e+00	0.7949700	0.681	0.495	0.0000000
MT-CO1	0.0e+00	0.3243268	0.989	0.979	0.0000000
TRGC1	0.0e+00	3.6853476	0.110	0.013	0.0000000
FHIT	0.0e+00	-1.6706990	0.110	0.273	0.0000000
RP11-291B21.2	0.0e+00	1.3507107	0.222	0.077	0.0000000
MT-ND4	0.0e+00	0.3226206	0.963	0.932	0.0000000
CD4	0.0e+00	-3.3247772	0.011	0.105	0.0000000
MT-CO2	0.0e+00	0.2401784	0.993	0.995	0.0000000
TRDC	0.0e+00	2.6894608	0.105	0.019	0.0000000
CRTAM	0.0e+00	3.7124154	0.068	0.003	0.0000000
PECAM1	0.0e+00	2.6701767	0.096	0.015	0.0000000
LYAR	0.0e+00	1.6580580	0.198	0.076	0.0000000
GZMH	0.0e+00	4.8543018	0.067	0.003	0.0000000
PRF1	0.0e+00	2.8596139	0.099	0.018	0.0000000
ACTB	0.0e+00	0.3093718	0.965	0.925	0.0000000
AC092580.4	0.0e+00	1.3359936	0.173	0.061	0.0000000
CCL4	0.0e+00	3.7724347	0.118	0.031	0.0000001
RPS27A	0.0e+00	-0.1605123	1.000	1.000	0.0000001
RPL11	0.0e+00	-0.1685948	1.000	1.000	0.0000002
KLRC1	0.0e+00	8.3089065	0.048	0.000	0.0000004
MT-CO3	0.0e+00	0.2753652	0.966	0.954	0.0000005
TPST2	0.0e+00	1.8352805	0.137	0.045	0.0000009

Differential expression using limma

The Wilcoxon test implemented in FindMarkers does not allow you to test for complex design (eg factorial experiments) or to include batch as a covariate. It doesn't allow you to run paired-sample T tests for example.

For more complex designs, we can use edgeR or limma which are designed for microarray or bulk RNA seq data and provide a design matrix that includes covariates for example, or sample IDs for paired analyses.

We will load an object containing only pro B cells, both from healthy tissues (PBMMC), and malignant tissues (ETV6-RUNX1).

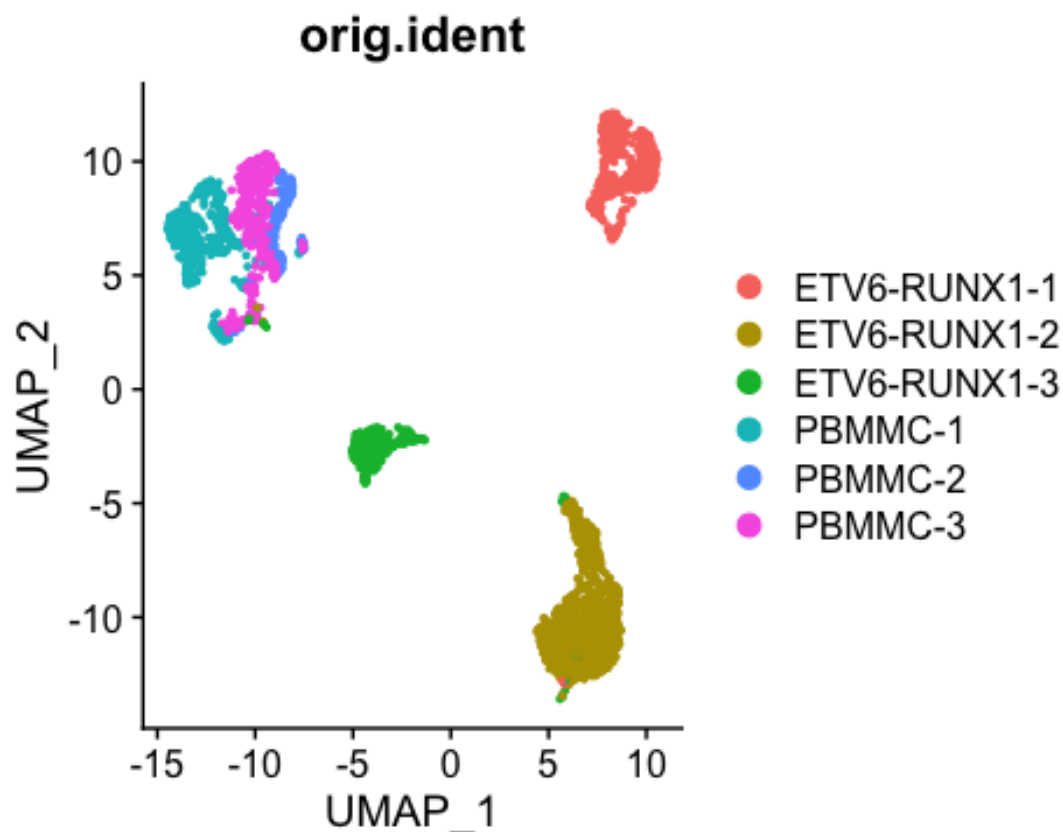
Warning

Please NOTE that in the original design of this data set, the healthy and malignant tissues were not patient-matched, i.e. the real design was not the one of paired healthy and malignant tissues. However, for demonstration purposes, we will show you how to run a paired analysis, and do as if the PBMMC-1 and ETV6-RUNX1-1 samples both came from the same patient 1, the PBMMC-2 and ETV6-RUNX1-2 samples both came from the same patient 2, etc...

We can load the object and explore its UMAP and meta.data like this:

Code starts here:

```
proB <- readRDS("course_data/proB.rds")  
Seurat::DimPlot(proB, group.by = "orig.ident")
```



Code starts here:

```
table(proB@meta.data$type)
```

```
ETV6-RUNX1      PBMMC
      2000      1021
```

```
head(proB@meta.data)
```

	orig.ident	nCount_RNA	nFeature_RNA	
SingleR_annot				
PBMMC-1_AAATGCCAGACTGGGT-1	PBMMC-1	4886	1727	Pro-
B_cell_CD34+				
PBMMC-1_AAATGCCTCCACTGGG-1	PBMMC-1	8397	2291	Pro-
B_cell_CD34+				
PBMMC-1_AACACGTTCTTGACGA-1	PBMMC-1	3444	1204	Pro-
B_cell_CD34+				
PBMMC-1_AACCATGAGAAGGTGA-1	PBMMC-1	8981	2437	Pro-
B_cell_CD34+				
PBMMC-1_AACCGCGCATGGTCAT-1	PBMMC-1	3719	1368	Pro-
B_cell_CD34+				
PBMMC-1_AAGCCGCCAGACGTAG-1	PBMMC-1	4573	1464	Pro-
B_cell_CD34+				

	type
PBMMC-1_AAATGCCAGACTGGGT-1	PBMMC
PBMMC-1_AAATGCCTCCACTGGG-1	PBMMC
PBMMC-1_AACACGTTCTTGACGA-1	PBMMC
PBMMC-1_AACCATGAGAAGGTGA-1	PBMMC
PBMMC-1_AACCGCGCATGGTCAT-1	PBMMC
PBMMC-1_AAGCCGCCAGACGTAG-1	PBMMC

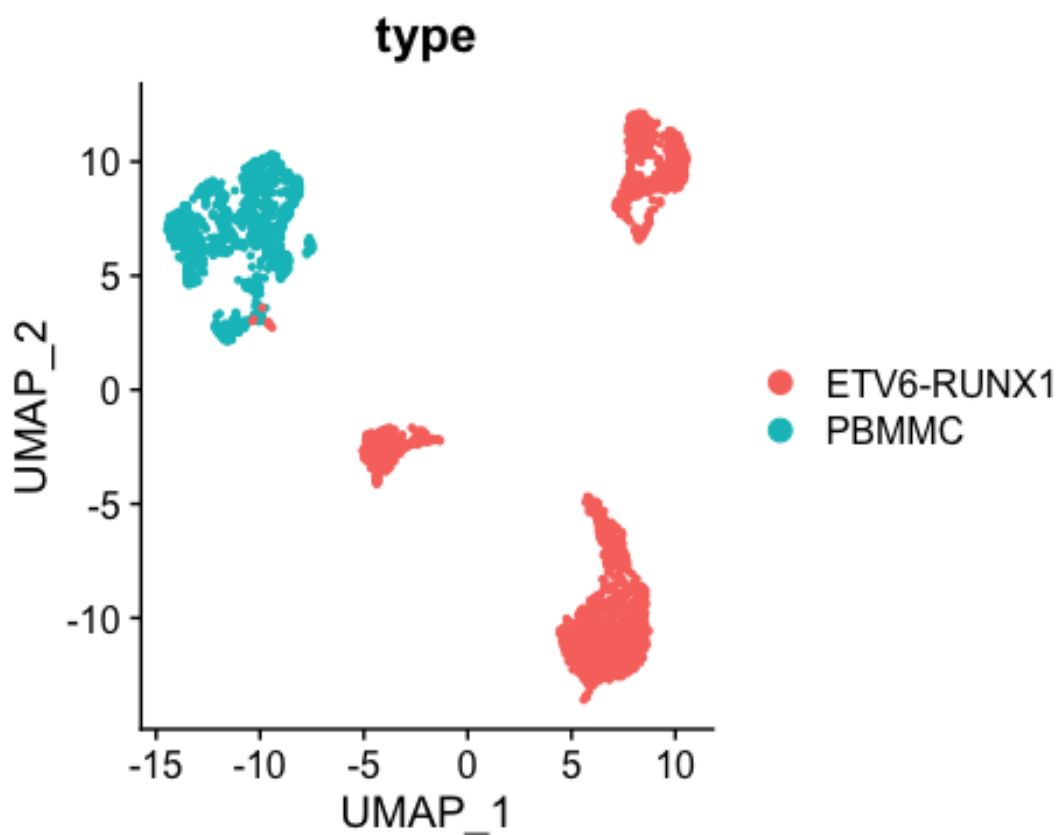
i Note

If you want to know how this pro-B cell subset is generated, have a look at the script [here](#).

Let's have a look at the UMAP (again), coloured by celltype:

Code starts here:

```
Seurat::DimPlot(proB, group.by = "type")
```



Let's say we are specifically interested to test for differential gene expression between the tumor and normal samples.

i Note

Here we could also test for e.g. healthy versus diseased within a celltype/cluster.

Now we will run differential expression analysis between tumor and healthy cells using the patient ID as a covariate by using limma.

Prepare the pseudobulk count matrix:

Code starts here:

```
#taking the proB data
Seurat::DefaultAssay(proB) <- "RNA"
Seurat::Idents(proB) <- proB$orig.ident

## add the patient id also for paired DGE
proB$patient.id<-gsub("ETV6-RUNX1", "ETV6_RUNX1", proB$orig.ident)
proB$patient.id<-sapply(strsplit(proB$patient.id, "-"), '[', 2)

## Here we do perform pseudo-bulk:
##first a mandatory column of sample needs to be added to the meta data that
is the grouping factor, should be the samples
proB$sample <- factor(proB$orig.ident)

# aggregate the cells per sample
bulk <- Seurat::AggregateExpression(proB, group.by = "sample",
                                   return.seurat = TRUE,
                                   assay = "RNA")

# create a metadata data frame based on the aggregated cells
meta_data <- unique(proB@meta.data[, c("orig.ident",
                                       "sample", "type",
                                       "patient.id")])

rownames(meta_data) <- meta_data$orig.ident
bulk@meta.data <- meta_data[colnames(bulk), ]

##have a look at the counts
counts <- Seurat::GetAssayData(bulk, layer = "counts") |> as.matrix()

head(counts)
```

	ETV6-RUNX1-1	ETV6-RUNX1-2	ETV6-RUNX1-3	PBMMC-1	PBMMC-2	PBMMC-3
RP11-34P13.7	0	0	0	2	0	0
F0538757.3	0	0	0	0	0	0
F0538757.2	138	275	74	129	40	112
AP006222.2	63	43	17	38	19	26
RP4-669L17.10	5	10	3	0	1	1
RP5-857K21.4	0	0	0	0	0	2

```
#have a look at the colData of our new object summed, can you see type and
#patient.id are there
head(bulk@meta.data)
```

	orig.ident	sample	type	patient.id
ETV6-RUNX1-1	ETV6-RUNX1-1	ETV6-RUNX1-1	ETV6-RUNX1	1
ETV6-RUNX1-2	ETV6-RUNX1-2	ETV6-RUNX1-2	ETV6-RUNX1	2
ETV6-RUNX1-3	ETV6-RUNX1-3	ETV6-RUNX1-3	ETV6-RUNX1	3
PBMMC-1	PBMMC-1	PBMMC-1	PBMMC	1
PBMMC-2	PBMMC-2	PBMMC-2	PBMMC	2
PBMMC-3	PBMMC-3	PBMMC-3	PBMMC	3

Generate a DGEList object to use as input for limma and filter the genes to remove lowly expressed genes. How many are left?

Code starts here:

```
#As in the standard limma analysis generate a DGE object
```

```
y <- edgeR::DGEList(counts, samples = bulk@meta.data)
```

```
##filter lowly expressed (recommanded for Limma)
keep <- edgeR::filterByExpr(y, group = bulk$type)
y <- y[keep,]
```

```
##see how many genes were kept
summary(keep)
```

Mode	FALSE	TRUE
logical	11086	10017

Generate a design matrix, including patient ID to model for a paired analysis. If you need help to generate a design matrix, check out the very nice [edgeR User Guide](#), sections 3.3 and 3.4. Extract the sample ID from the meta.data, then create the design matrix:

Code starts here:

```
## Create the design matrix and include the technology as a covariate:
design <- model.matrix(~0 + y$samples$type + y$samples$patient.id)
```

```
# Have a look
design
```

	y\$samples\$typeETV6-RUNX1	y\$samples\$typePBMMC	y\$samples\$patient.id2
1	1	0	0
2	1	0	1
3	1	0	0
4	0	1	0
5	0	1	1
6	0	1	0

y\$samples\$patient.id3

```

1          0
2          0
3          1
4          0
5          0
6          1
attr(,"assign")
[1] 1 1 2 2
attr(,"contrasts")
attr(,"contrasts")$`y$samples$type`
[1] "contr.treatment"

attr(,"contrasts")$`y$samples$patient.id`
[1] "contr.treatment"

# change column/rownames names to more simple group names:
colnames(design) <- make.names(c("ETV6-RUNX1",
"PBMHC","patient2","patient3"))
rownames(design) <- rownames(y$samples)

```

Specify which contrast to analyse:

Code starts here:

```

contrast.mat <- limma::makeContrasts(ETV6.RUNX1 - PBMHC,
                                   levels = design)

```

First, we perform TMM normalization using edgeR, and then limma can perform the transformation with voom, fit the model, compute the contrasts and compute test statistics with eBayes:

Code starts here:

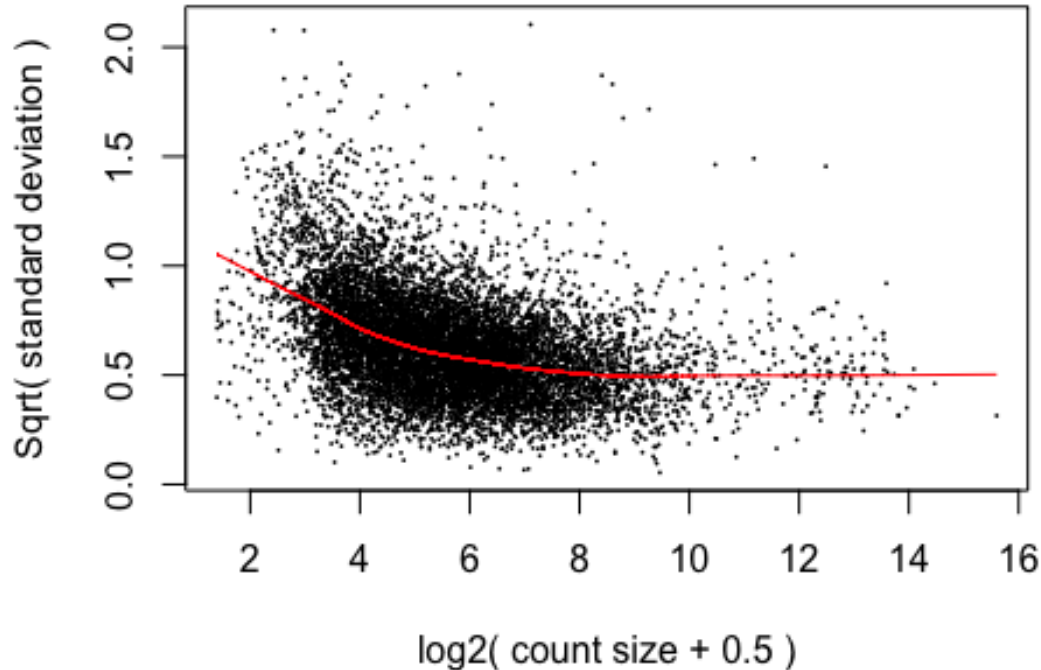
```

dge <- edgeR::calcNormFactors(y)

#Do limma
vm <- limma::voom(dge, design = design, plot = TRUE)

```

voom: Mean-variance trend



Code starts here:

```
fit <- limma::lmFit(v, design = design)
fit.contrasts <- limma::contrasts.fit(fit, contrast.mat)
fit.contrasts <- limma::eBayes(fit.contrasts)
```

We can use `topTable` to get the most significantly differentially expressed genes, and save the full DE results to an object. How many genes are significant? Are you surprised by this number?

Code starts here:

```
# Show the top differentially expressed genes:
limma::topTable(fit.contrasts, number = 10, sort.by = "P")
```

	logFC	AveExpr	t	P.Value	adj.P.Val	B
RPS4Y2	5.346800	6.347826	15.39674	3.361453e-08	0.0001152727	9.477129
SDC2	9.070465	2.708711	15.33434	3.493119e-08	0.0001152727	7.681010
IGLL1	-3.788160	9.287148	-15.19483	3.808426e-08	0.0001152727	9.465422
CTGF	4.368363	6.029640	14.89301	4.603081e-08	0.0001152727	9.141505
AP005530.2	8.770808	2.560369	14.27130	6.879238e-08	0.0001335825	7.294079
GNG11	3.500250	6.457777	13.70183	1.008304e-07	0.0001335825	8.495288
HLA-DQA1	2.982748	7.410939	13.39202	1.249067e-07	0.0001335825	8.287394
PTP4A3	3.734865	5.449894	13.23439	1.395246e-07	0.0001335825	8.126807

```
CD27      4.115561 5.843582 13.23011 1.399471e-07 0.0001335825 8.147433
ALOX5     4.142010 5.682900 13.16658 1.463814e-07 0.0001335825 8.098199
```

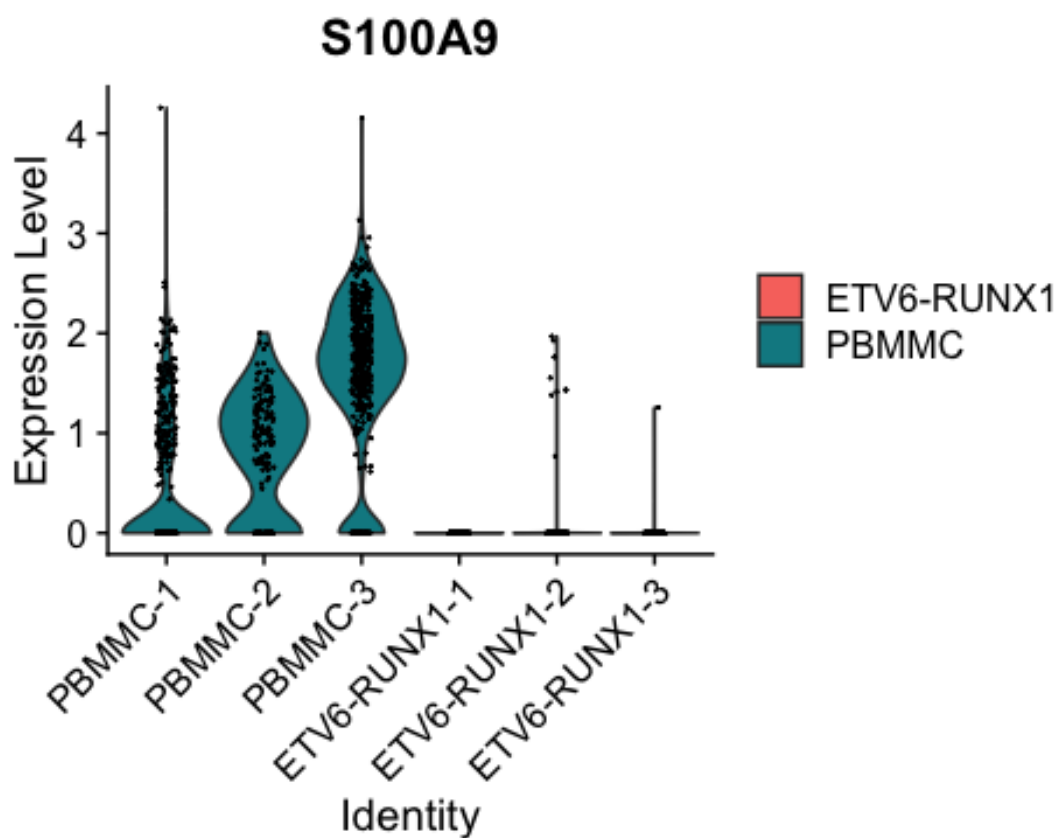
```
limma_de <- limma::topTable(fit.contrasts, number = Inf, sort.by = "P")
length(which(limma_de$adj.P.Val<0.05))
```

```
[1] 2738
```

And we can check whether this corresponds to the counts by generating a violin plot, or a gene downregulated in tumor, or a gene upregulated in tumor:

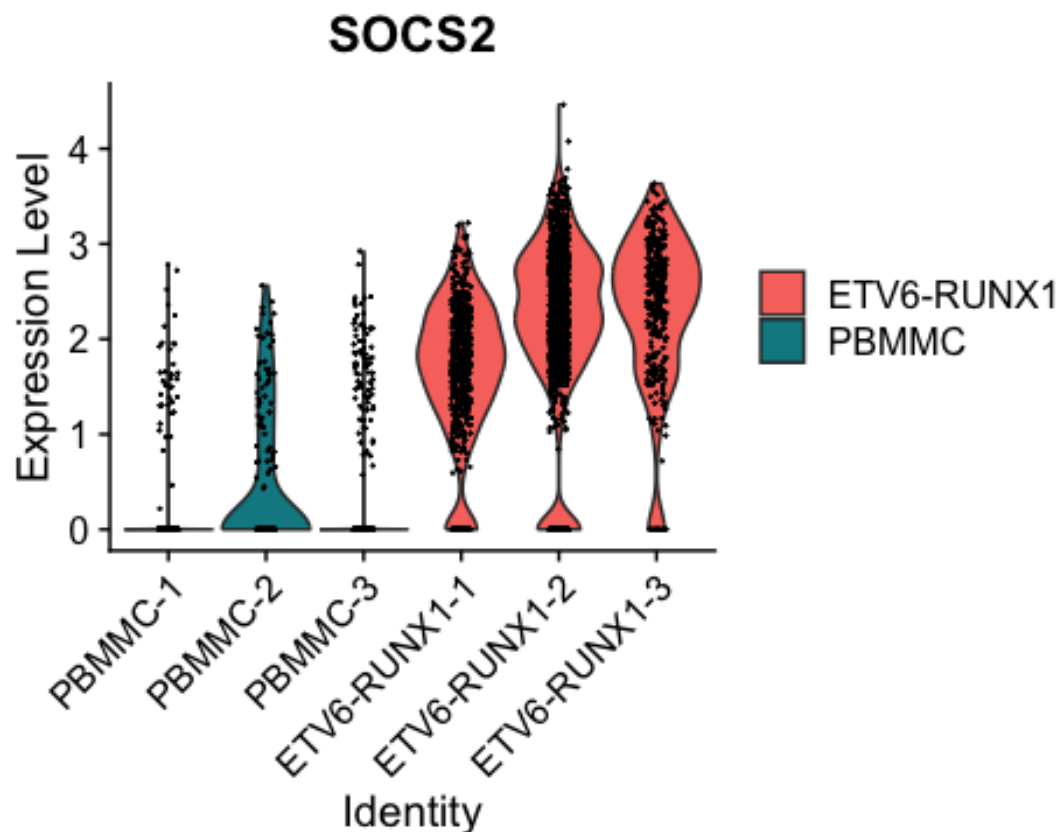
Code starts here:

```
Seurat::VlnPlot(proB, "S100A9", split.by = "type")
```



Code starts here:

```
Seurat::VlnPlot(proB, "SOCS2", split.by = "type")
```



We can run a similar analysis with Seurat, but this will not take into account the paired design. Run the code below.

Code starts here:

```
tum_vs_norm <- Seurat::FindMarkers(proB,
                                   ident.1 = "ETV6-RUNX1",
                                   ident.2 = "PBMMC",
                                   group.by = "type")
tum_vs_norm <- subset(tum_vs_norm, tum_vs_norm$p_val_adj < 0.05)
```

⚠ Exercise (extra)

How many genes are significant? How does the fold change of these genes compare to the fold change of the top genes found by limma?

💡 Answer

Code starts here:

```
dim(tum_vs_norm)
```

```
[1] 3820    5
```

We find 3820 significant genes. If we merge the FindMarkers and the limma results, keep limma's most significant genes and plot:

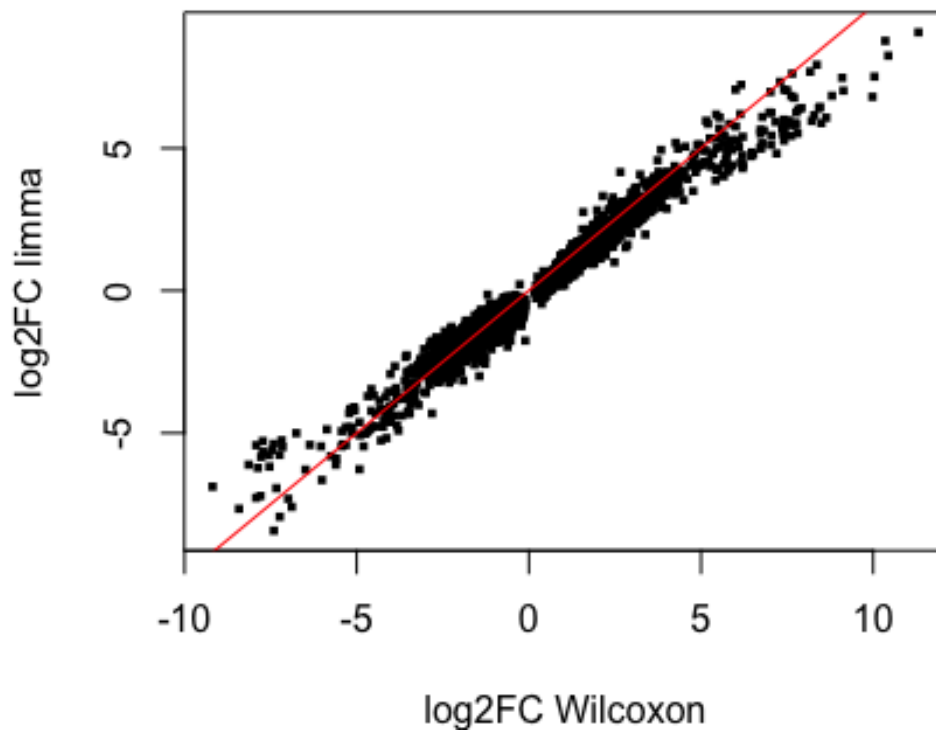
Code starts here:

```
merge_limma_FindMarkers <- merge(tum_vs_norm, limma_de, by="row.names",  
                                  all.x=T)
```

```
par(mar=c(4,4,4,4))
```

```
plot(merge_limma_FindMarkers$avg_log2FC,  
     merge_limma_FindMarkers$logFC,  
     xlab="log2FC Wilcoxon", ylab="log2FC limma",  
     pch=15, cex=0.5)
```

```
abline(a=0, b=1, col="red")
```



 Keep the object

Keep the `tum_vs_norm` and `limma_de` objects because we will use this output later for the enrichment analysis in the next section.